

课程设计报告

设计名称：_____ 基于 Android 的汽车租赁 APP

姓 名：_____ XXX _____ 学 号：_____ XXXXXXXXXXXXXXXX

专业班级：_____ XXXXXXXXXXXXXXXX

系（院）：_____ XXXXXXXX

设计时间：_____ XXXXXXXXXXXXXXXX

设计地点：_____ XXXXXXXX

指导教师评语：

成绩：

签名：_____

2024 年 1 月 19 日

1. 课程设计目的

《移动应用开发课程设计》是计算机科学与技术相关专业实践性环节之一, 是学习完《移动应用开发》课程后进行的一次全面的综合练习。其目的在于加深对 Android 开发基本知识的理解, 掌握使用 Java 语言进行 Android 应用开发的基本方法, 提高运用知识解决实际问题的能力。基本任务是使得学生通过课程设计掌握利用 Java 进行 Android 应用开发的基本方法和技能。

2. 课程设计任务与要求:

2.1 任务

2.1.1 可选任务说明

学生单独完成一个项目, 在教师提供的课程设计题目中任意选择一题, 独立完成, 题目选定后不可更换。学生也可自选项目任务课题, 须经指导老师同意。可供选择的题目范围具体见课程设计的计划书, 具体要求和解释见任务书。

2.1.2 选择的任务

选择的具体移动应用开发项目任务为: 基于 Android 的汽车租赁 APP

2.2 要求

2.2.1 设计要求

- (1) 分析课程设计题目的要求;
- (2) 写出课设任务的详细设计说明;
- (3) 编写程序代码, 调试程序使其能正确运行;
- (4) 设计完成后提交课程设计报告, 课程设计报告内容包括: 课程设计目的、设计任务与要求、设计说明书、设计成果和设计心得五个部分, 具体要求见设计报告模板。

2.2.3 提交要求

- (1) 每位同学需独立提交书面设计报告书(每人一份), 要求编排格式统一、规范、内容充实; 同时提交设计报告电子版。
- (2) 提交日期: 2024 年 01 月 19 日之前。
- (3) 提交方式: 文档统一命名为: 班级-学号-姓名_移动应用开发_课程设计报告

3. 课程设计说明书

3.1 课程设计目的

《移动应用开发课程设计》是计算机科学与技术相关专业实践性环节之一, 是学习完《移动应用开发》课程后进行的一次全面的综合练习。其目的在于加深对 Android 开发基本知识的理解, 掌握使用 Java 语言进行 Android 应用开发的基本方法, 提高运用知识解决实际问题的能力。基本任务是使得学生通过课程设计掌握利用 Java 进行 Android 应用开发的基本方法和技能。

3.2 课设需求

1、实验硬件设备:

电脑一台。

2、实验软件工具:

1) 开发环境

Android Studio 2025.1.3

2) 文档编辑工具

WPS Office

3.3 核心程序设计

3.3.1 功能模块设计

基于 MVC 架构, 整个汽车租赁 APP 被系统性地划分为模型 (Model)、视图 (View) 和控制器 (Controller) 三个层次, 以实现关注点分离、提高代码可维护性和可扩展性。各层之间职责清晰, 通过明确定义的接口进行通信, 确保模块间耦合度低、内聚性高。具体设计如下:

1. 模型层 (Model Layer)

模型层负责数据管理和业务逻辑, 是应用程序的核心数据处理单元。

实体模型: 定义应用程序的核心数据结构。

User: 用户实体, 包含用户 ID、用户名、密码 (加密后)、手机号、邮箱等属性。

Car: 车辆实体, 包含车辆 ID、品牌、型号、车牌号、日租金、状态 (可租/已租)、图片资源 ID、详细描述等属性。

Order: 订单实体, 包含订单 ID、关联用户 ID、关联车辆 ID、取车时间、还车时间、取车地点、还车地点、订单总价、订单状态 (待确认/进行中/已完成) 等属性。

数据访问层:

DatabaseHelper: 继承自 SQLiteOpenHelper, 负责数据库的创建、版本升级。

UserDAO、CarDAO、OrderDAO: 数据访问对象, 封装对 User、Car、Order 表的增、删、改、查 (CRUD) 操作, 所有数据库操作均在子线程中执行。

业务逻辑: 在 DAO 层或独立的服务类中封装核心计算逻辑, 如根据租期和日租金计算订单总费用、验证用户登录信息、检查车辆在特定时段内的可租状态等。

2. 视图层 (View Layer)

视图层负责用户界面的展示和用户交互的捕获, 由一系列 Activity 和 Fragment 及其对应的 XML 布局文件构成。

Activity/Fragment: 作为视图的容器和控制器的一部分 (在标准 Android MVC 中)。

LoginActivity / RegisterActivity: 用户登录和注册界面。

MainActivity: 主界面, 通常包含底部导航, 承载车辆列表、个人中心等 Fragment。

CarListFragment: 车辆列表展示页面, 使用 RecyclerView 和 CardView。

CarDetailActivity: 车辆详细信息展示页面。

BookingActivity: 租车预订流程页面。

OrderListActivity / OrderDetailActivity: 订单列表和详情页面。

ProfileActivity: 个人中心信息查看与编辑页面。

布局与控件:

使用 ConstraintLayout、LinearLayout 等构建响应式界面。

采用 RecyclerView 高效展示车辆和订单列表，CardView 使条目呈现卡片式立体效果。

使用 EditText、Button、ImageView 等标准控件实现交互元素。

职责：严格限于显示由控制器提供的数据，并将用户输入（如按钮点击、表单提交）事件传递给控制器处理。

3. 控制层（Controller Layer）

控制层作为模型与视图之间的桥梁，负责处理业务逻辑、响应用户输入并更新视图。

主要载体：在 Android 开发中，Activity 和 Fragment 通常承担了控制器的主要角色。

核心职责：

事件监听与路由：监听视图层触发的各类事件（如 onClick），并根据事件类型调用相应的模型层方法或启动新的视图。

数据中介：从模型层（DAO）获取数据，进行处理或格式化后，传递给视图层进行展示；同时，将视图层收集的用户输入数据验证后传递给模型层进行持久化。

状态管理：在界面旋转或前后台切换时，负责保存和恢复必要的界面状态。

3.3.2 数据库设计

本系统数据库 CarRental.db 包含三个核心表，其详细设计如下：

表 1: 用户表 (users)				
字段名	数据类型	数据长度	约束条件	说明
user_id	INTEGER	-	PRIMARY KEY AUTOINCREMENT	用户唯一标识
username	TEXT	20	UNIQUE, NOT NULL	用户名
password	TEXT	64	NOT NULL	密码（预留哈希值长度）
email	TEXT	50	-	电子邮箱
phone	TEXT	15	-	手机号码
full_name	TEXT	50	-	用户全名

表 2: 汽车表 (cars)				
字段名	数据类型	数据长度	约束条件	说明
car_id	INTEGER	-	PRIMARY KEY AUTOINCREMENT	汽车唯一标识
brand	TEXT	20	NOT NULL	汽车品牌
model	TEXT	30	NOT NULL	汽车型号
price_per_day	REAL	(10,2)	NOT NULL	每日租金（元），10 位总长，2 位小数
description	TEXT	200	-	汽车描述
image_url	TEXT	50	-	图片资源标识符

is_available	INTEGER	-	DEFAULT 1	是否可用（1-是，0-否）
--------------	---------	---	-----------	---------------

表 3：订单表 (orders)

字段名	数据类型	数据长度	约束条件	说明
order_id	INTEGER	-	PRIMARY KEY AUTOINCREMENT	订单唯一标识
user_id	INTEGER	-	NOT NULL, FOREIGN KEY	用户 ID，关联 users 表
car_id	INTEGER	-	NOT NULL, FOREIGN KEY	汽车 ID，关联 cars 表
start_date	INTEGER	-	NOT NULL	租赁开始时间（Unix 时间戳）
end_date	INTEGER	-	NOT NULL	租赁结束时间（Unix 时间戳）
pickup_location	TEXT	100	NOT NULL	取车地点
return_location	TEXT	100	NOT NULL	还车地点
total_price	REAL	(12,2)	NOT NULL	订单总金额，12 位总长，2 位小数
status	TEXT	20	DEFAULT 'pending'	订单状态

3.4 课程设计步骤

3.4.1 功能性需求

本系统主要涉及两类用户角色：普通用户和系统。普通用户是系统的核心使用者。

根据用例图，可以将系统核心功能模块分解如下：

用户管理模块：负责处理用户账户相关的所有操作。

车辆信息管理模块：负责车辆数据的存储、查询与展示。

预订与订单管理模块：负责处理用户的租赁预订请求，生成并管理订单。

个人中心模块：为用户提供信息查看与编辑的入口。

系统的功能需求可以通过以下用例图来描述：

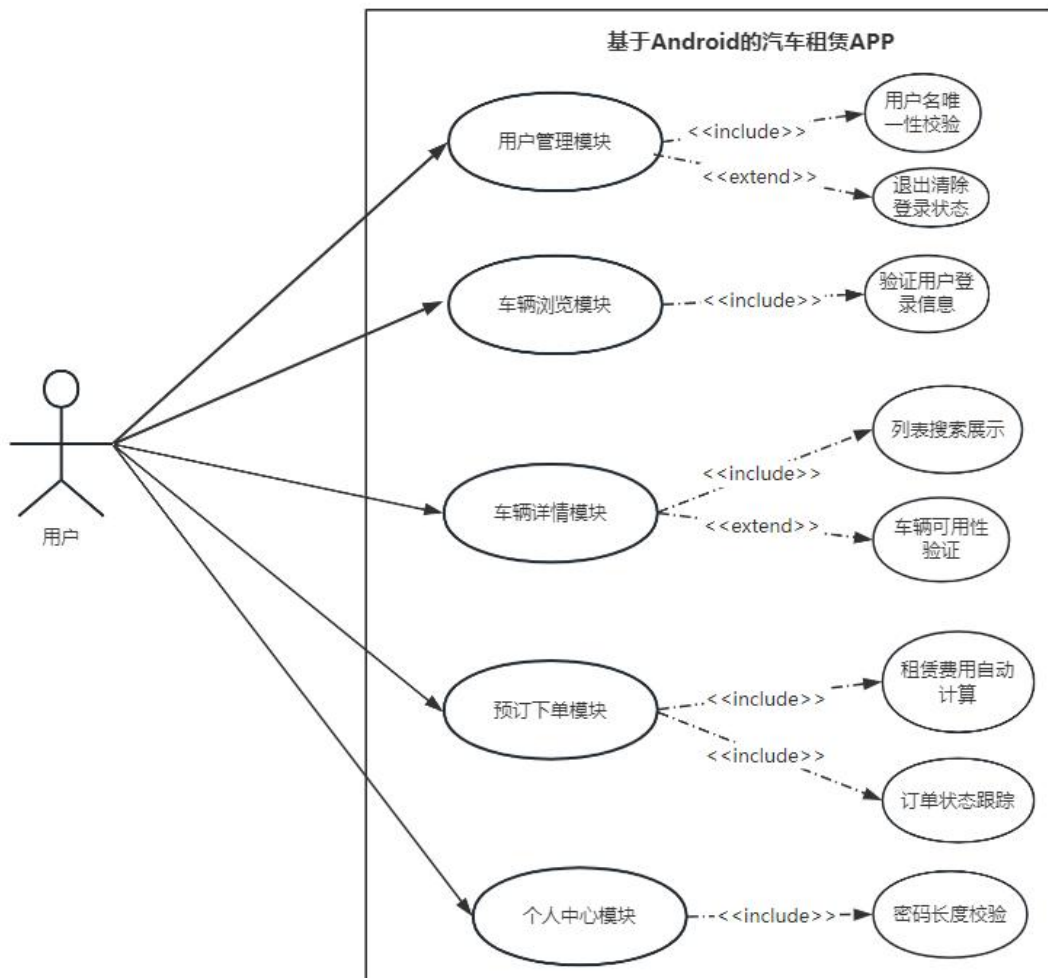


图 3-1：系统总体用例图

3.1.2 非功能性需求

用户界面：界面设计应遵循 Material Design 原则，确保在不同屏幕尺寸的 Android 设备上均有良好的显示效果。

数据一致性：确保车辆的可租状态与订单状态的一致性，避免出现超售情况。

安全性：对用户输入的敏感信息进行校验，防止 SQL 注入等基本安全威胁。

3.2 概要设计

3.2. 功能模块划分

系统的功能模块结构图如下：

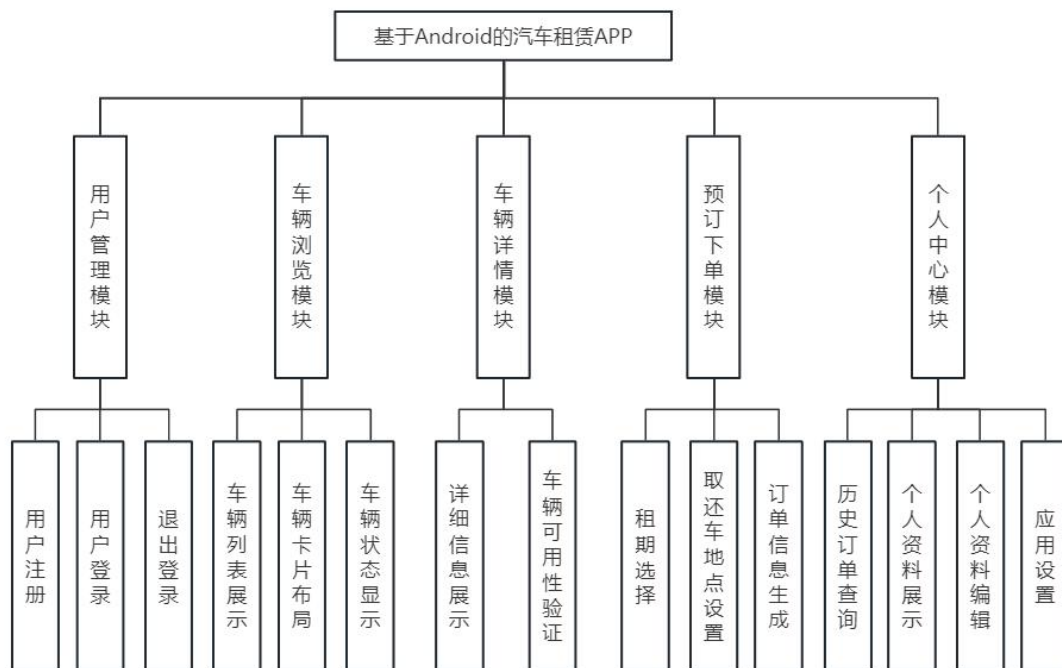


图 3-2: 系统功能模块图

3.2.3 类结构设计

根据需求分析，系统中需要设计以下核心类：

模型类：

User：用户数据模型，封装用户基本信息。

Car：汽车数据模型，封装汽车相关信息。

Order：订单数据模型，封装订单详细信息。

数据库操作类：

DatabaseHelper：继承自 SQLiteOpenHelper，负责数据库的创建、升级以及所有对用户、汽车、订单表的增、删、改、查操作。

界面控制类：

WelcomeActivity：欢迎界面，应用入口。

LoginActivity：用户登录界面。

RegisterActivity：用户注册界面。

MainActivity：主界面，展示汽车列表和搜索功能。

CarDetailActivity：汽车详情界面。

ReservationActivity：车辆预订界面。

OrderConfirmationActivity：订单确认与支付界面。

ProfileActivity：个人资料与订单历史界面。

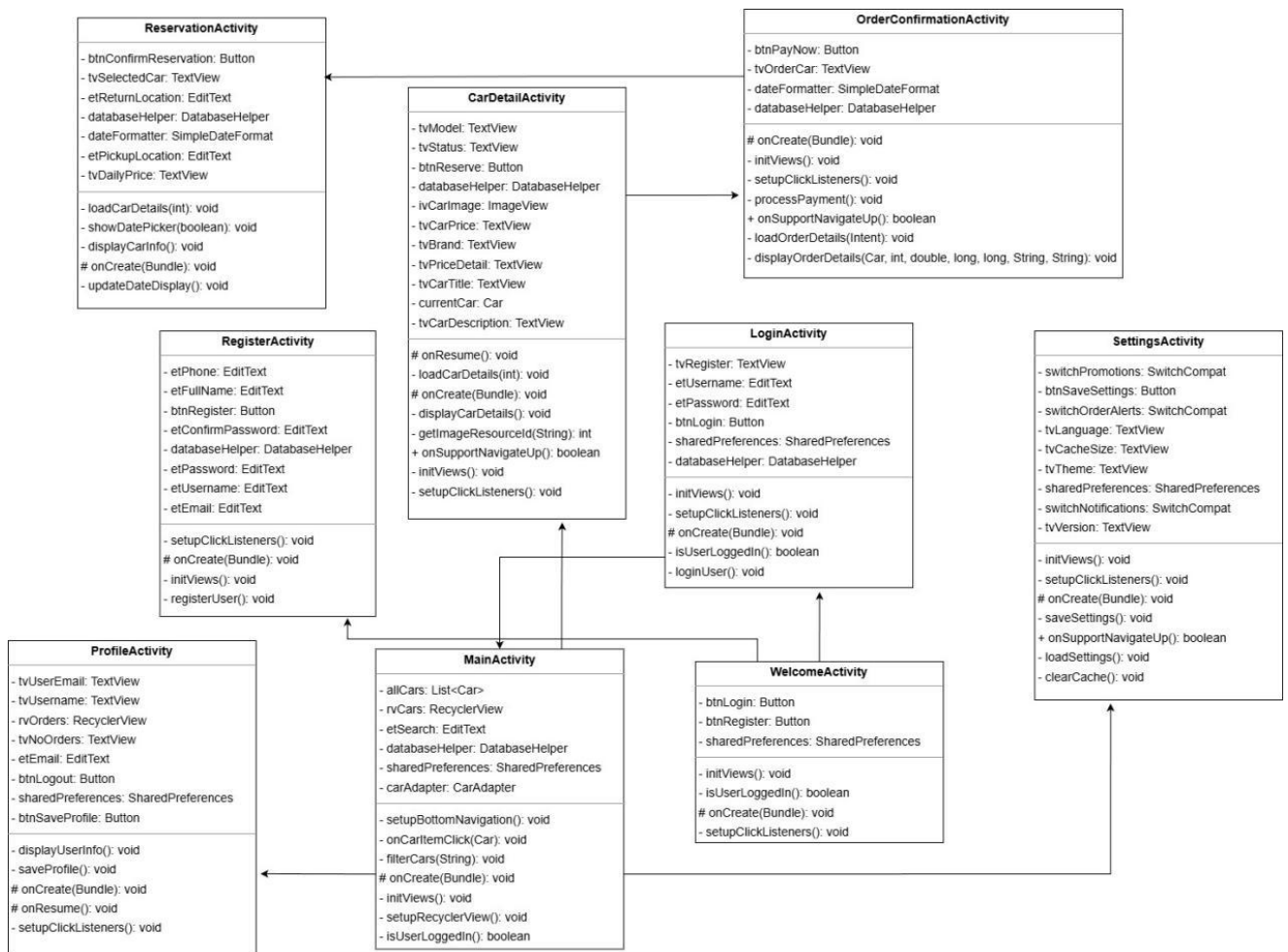
SettingsActivity：系统设置界面。

适配器类：

CarAdapter：用于 MainActivity 中 RecyclerView 的汽车列表适配。

OrderAdapter：用于 ProfileActivity 中 RecyclerView 的订单列表适配。

本系统类图如下所示:



3.3 详细设计

详细设计部分将深入说明系统中各个核心类的职责、方法以及它们之间的协作关系。

3.3.1 核心类与方法设计

1. 模型类 (Model Classes)

User 类

职责: 封装用户的基本信息。

主要属性: id, username, password, email, phone, fullName。

主要方法: 所有属性的 Getter 和 Setter 方法。

Car 类

职责: 封装汽车的各项属性。

主要属性: id, brand, model, pricePerDay, description, imageUrl, isAvailable。

主要方法: 所有属性的 Getter 和 Setter 方法。

Order 类

职责: 封装订单的完整信息。

主要属性: id, userId, carId, startDate, endDate, pickupLocation, returnLocation, totalPrice, status。

主要方法: 所有属性的 Getter 和 Setter 方法。

2. 数据库辅助类 (Database Helper Class)

DatabaseHelper 类

职责: 管理 SQLite 数据库的创建、版本升级, 并提供所有数据表的核心操作方法。

继承关系: 继承自 SQLiteOpenHelper。

核心方法:

onCreate(SQLiteDatabase db): 创建用户表、汽车表、订单表, 并初始化示例汽车数据。

onUpgrade(...): 处理数据库版本升级策略。

addUser(User user): 向用户表插入新用户。

checkUser(String username, String password): 验证用户登录信息。

getUser(String username), getUserById(int userId): 根据用户名或 ID 获取用户信息。

isUsernameExists(String username): 检查用户名是否已存在。

updateUser(User user): 更新用户信息。

getAllCars(): 获取所有可用车辆的列表。

getCarById(int carId): 根据 ID 获取特定车辆信息。

searchCars(String keyword): 根据关键词搜索车辆。

addOrder(Order order): 创建新订单。

getUserOrders(int userId), getUserOrdersWithCarInfo(int userId): 获取指定用户的所有订单列表。

3. 适配器类 (Adapter Classes)

CarAdapter 类

职责: 作为 MainActivity 中 RecyclerView 的数据适配器, 将 Car 数据列表中的每一项数据与对应的列表项视图进行绑定。

核心方法:

onCreateViewHolder(): 创建列表项视图。

onBindViewHolder(): 将数据绑定到列表项视图上。

getItemCount(): 返回数据项总数。

setCarList(List<Car> carList): 更新适配器中的数据集。

OrderAdapter 类

职责: 作为 ProfileActivity 中 RecyclerView 的数据适配器, 用于展示用户的订单历史列表。

核心方法: 与 CarAdapter 类似, 包括 onCreateViewHolder, onBindViewHolder, getItemCount 等。

4. 活动类 (Activity Classes)

这些类继承自 AppCompatActivity, 主要负责控制界面生命周期和处理用户交互。

WelcomeActivity

职责: 应用入口, 展示欢迎界面, 提供登录和注册入口按钮。

核心方法: onCreate 初始化界面并设置按钮点击事件。

LoginActivity

职责：处理用户登录逻辑。

核心方法：onCreate 初始化界面，onClickListener 处理登录按钮点击事件（调用 DatabaseHelper.checkUser 进行验证）。

RegisterActivity

职责：处理新用户注册逻辑。

核心方法：onCreate 初始化界面，onClickListener 处理注册按钮点击事件（调用 DatabaseHelper.isUsernameExists 和 DatabaseHelper.addUser）。

MainActivity

职责：应用主界面，展示汽车列表，集成搜索功能。

核心方法：onCreate 初始化 RecyclerView 和 CarAdapter，并调用 DatabaseHelper.getAllCars() 或 searchCars() 加载数据。

CarDetailActivity

职责：展示选定汽车的详细信息，并提供“立即预订”入口。

核心方法：onCreate 通过 Intent 获取传入的 Car ID，调用 DatabaseHelper.getCarById 获取数据并填充界面。

ReservationActivity

职责：处理车辆预订流程，包括选择日期、地点，计算费用。

核心方法：onCreate 初始化界面（如日期选择器），onClickListener 处理预订按钮点击事件（进行输入验证并调用 DatabaseHelper.addOrder）。

OrderConfirmationActivity

职责：展示订单最终信息，提供模拟支付选项。

核心方法：onCreate 显示订单详情，onClickListener 处理不同支付方式的选择。

ProfileActivity

职责：展示和编辑用户个人信息，并列出历史订单。

核心方法：onCreate 加载用户信息和订单列表（调用 DatabaseHelper.getUserById 和 getUserOrdersWithCarInfo），onClickListener 处理信息保存和退出登录。

SettingsActivity

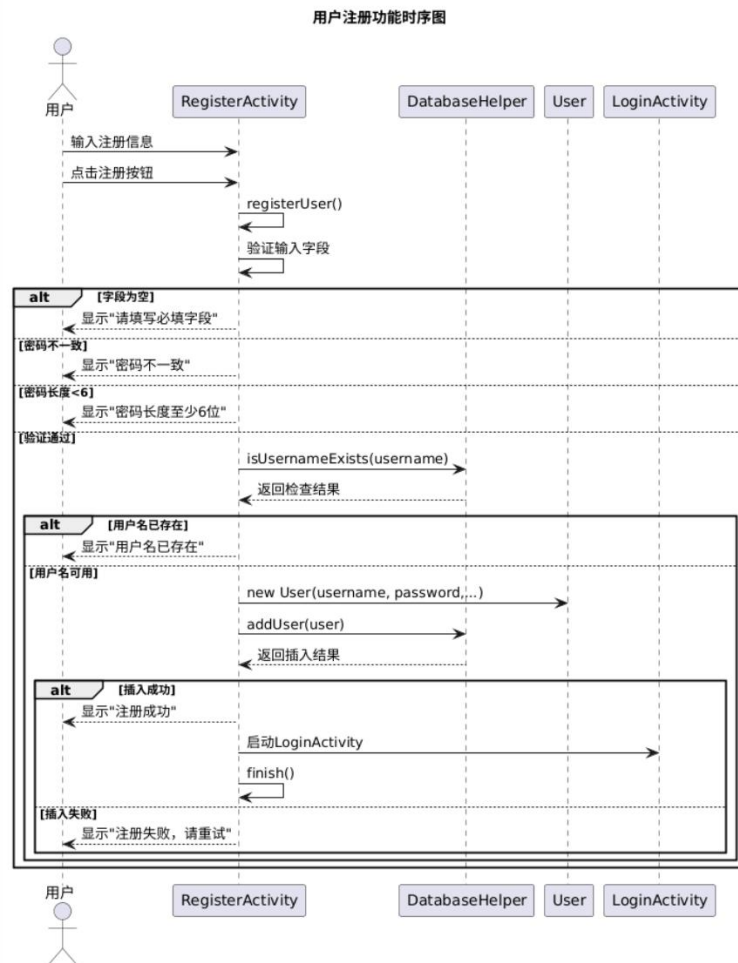
职责：提供应用设置选项。

核心方法：onCreate 初始化各项设置。

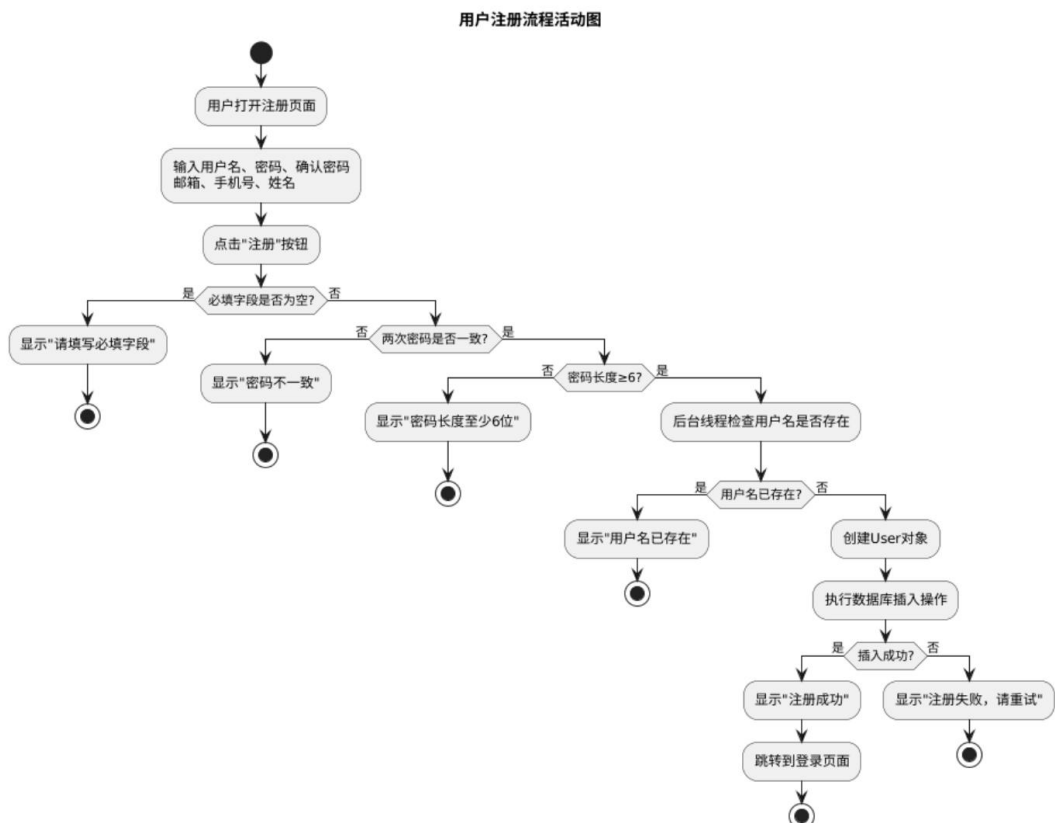
3.3.2 核心功能实现

(1) 用户注册功能

① 用户注册功能时序图



②用户注册流程活动图



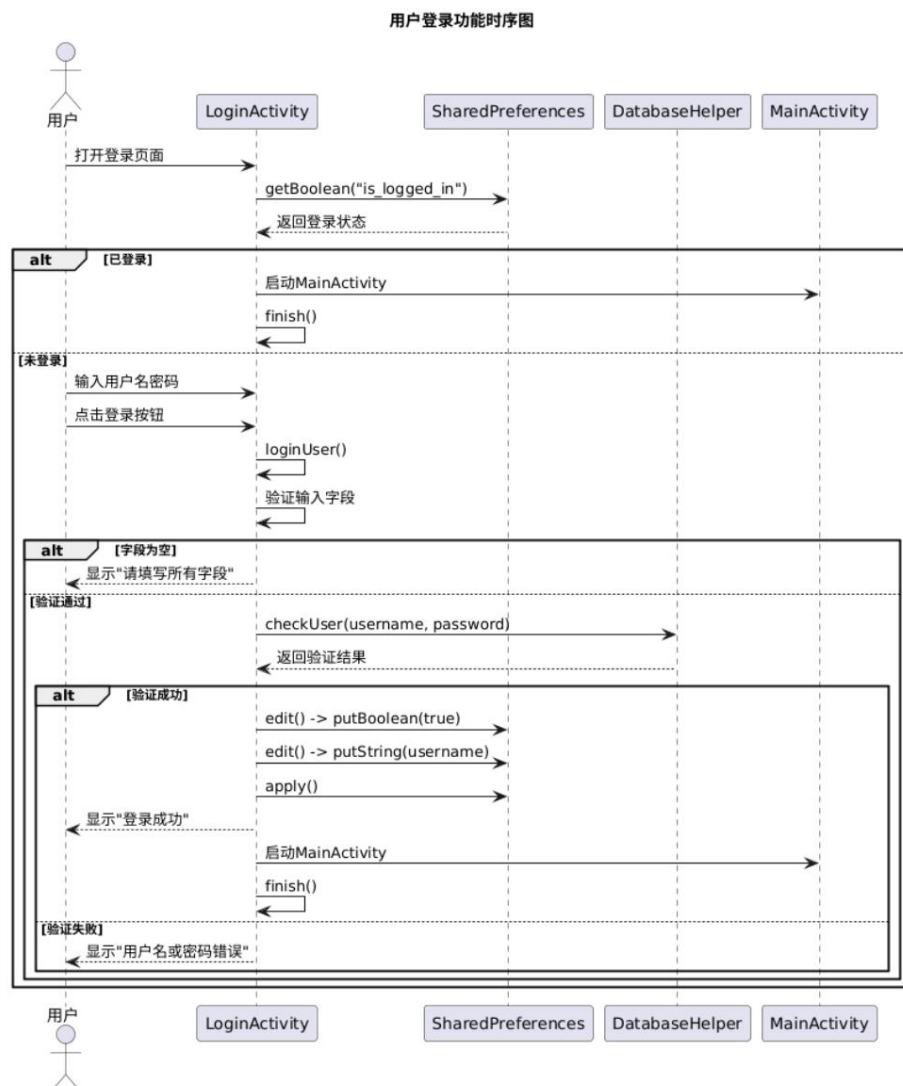
③核心代码

```
12 public class RegisterActivity extends AppCompatActivity {
    2 usages
13     private EditText etUsername, etPassword, etConfirmPassword, etEmail, etPhone, etFullName;
    2 usages
14     private Button btnRegister;
    3 usages
15     private DatabaseHelper databaseHelper;
16
17     @Override
18     protected void onCreate(Bundle savedInstanceState) {
19         super.onCreate(savedInstanceState);
20         setContentView(R.layout.activity_register);
21
22         initView();
23         databaseHelper = new DatabaseHelper(context: this);
24         setupClickListeners();
25     }
26
    1 usage
27     private void initView() {
28         etUsername = findViewById(R.id.etUsername);
29         etPassword = findViewById(R.id.etPassword);
30         etConfirmPassword = findViewById(R.id.etConfirmPassword);
31         etEmail = findViewById(R.id.etEmail);
32         etPhone = findViewById(R.id.etPhone);
33         etFullName = findViewById(R.id.etFullName);
34         btnRegister = findViewById(R.id.btnRegister);
35     }
36
    1 usage
37     private void setupClickListeners() { btnRegister.setOnClickListener(View v -> registerUser()); }
40
    1 usage
41     private void registerUser() {
42         String username = etUsername.getText().toString().trim();
43         String password = etPassword.getText().toString().trim();
44         String confirmPassword = etConfirmPassword.getText().toString().trim();
45         String email = etEmail.getText().toString().trim();
46         String phone = etPhone.getText().toString().trim();
47         String fullName = etFullName.getText().toString().trim();
48
49         if (username.isEmpty() || password.isEmpty() || confirmPassword.isEmpty()) {
50             Toast.makeText(context: this, text: "请填写必填字段", Toast.LENGTH_SHORT).show();
51             return;
52         }
53
54         if (!password.equals(confirmPassword)) {
55             Toast.makeText(context: this, text: "密码不一致", Toast.LENGTH_SHORT).show();
56             return;
57         }
58
59         if (password.length() < 6) {
60             Toast.makeText(context: this, text: "密码长度至少6位", Toast.LENGTH_SHORT).show();
61             return;
62         }
}
```

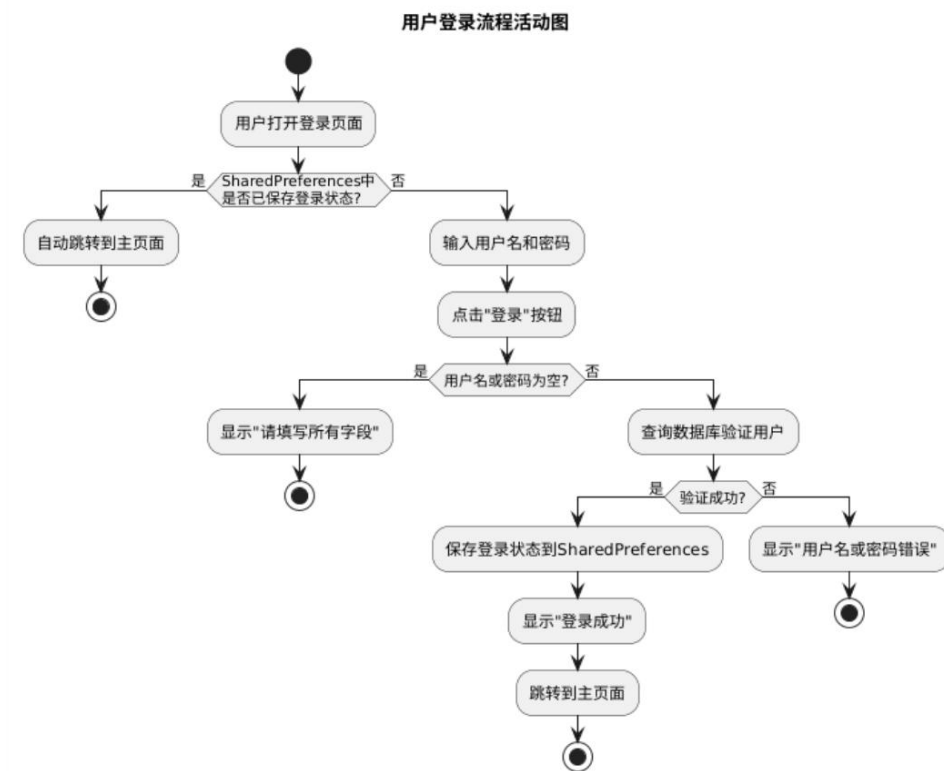
```
64 // 检查用户名是否存在
65 new Thread() -> {
66     boolean usernameExists = databaseHelper.isUsernameExists(username);
67     runOnUiThread() -> {
68         if (usernameExists) {
69             Toast.makeText( context: RegisterActivity.this, text: "用户名已存在", Toast.LENGTH_SHORT).show();
70         } else {
71             // 创建新用户
72             User user = new User(username, password, email, phone, fullName);
73
74             if (databaseHelper.addUser(user)) {
75                 Toast.makeText( context: RegisterActivity.this, text: "注册成功", Toast.LENGTH_SHORT).show();
76                 startActivity(new Intent( packageContext: RegisterActivity.this, LoginActivity.class));
77                 finish();
78             } else {
79                 Toast.makeText( context: RegisterActivity.this, text: "注册失败, 请重试", Toast.LENGTH_SHORT).show();
80             }
81         }
82     };
83 }.start();
84 }
```

(2) 用户登录功能

①用户登录功能时序图



②用户登录流程活动图



③核心代码实现

```
21 ⑥
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55

protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_login);

    initView();
    databaseHelper = new DatabaseHelper( context: this);
    sharedPreferences = getSharedPreferences( name: "user_prefs", MODE_PRIVATE);

    // 检查是否已登录
    if (isUserLoggedIn()) {
        startActivity(new Intent( packageContext: LoginActivity.this, MainActivity.class));
        finish();
        return;
    }

    setupClickListeners();
}

1 usage
private void initView() {
    etUsername = findViewById(R.id.etUsername);
    etPassword = findViewById(R.id.etPassword);
    btnLogin = findViewById(R.id.btnLogin);
    tvRegister = findViewById(R.id.tvRegister);
}

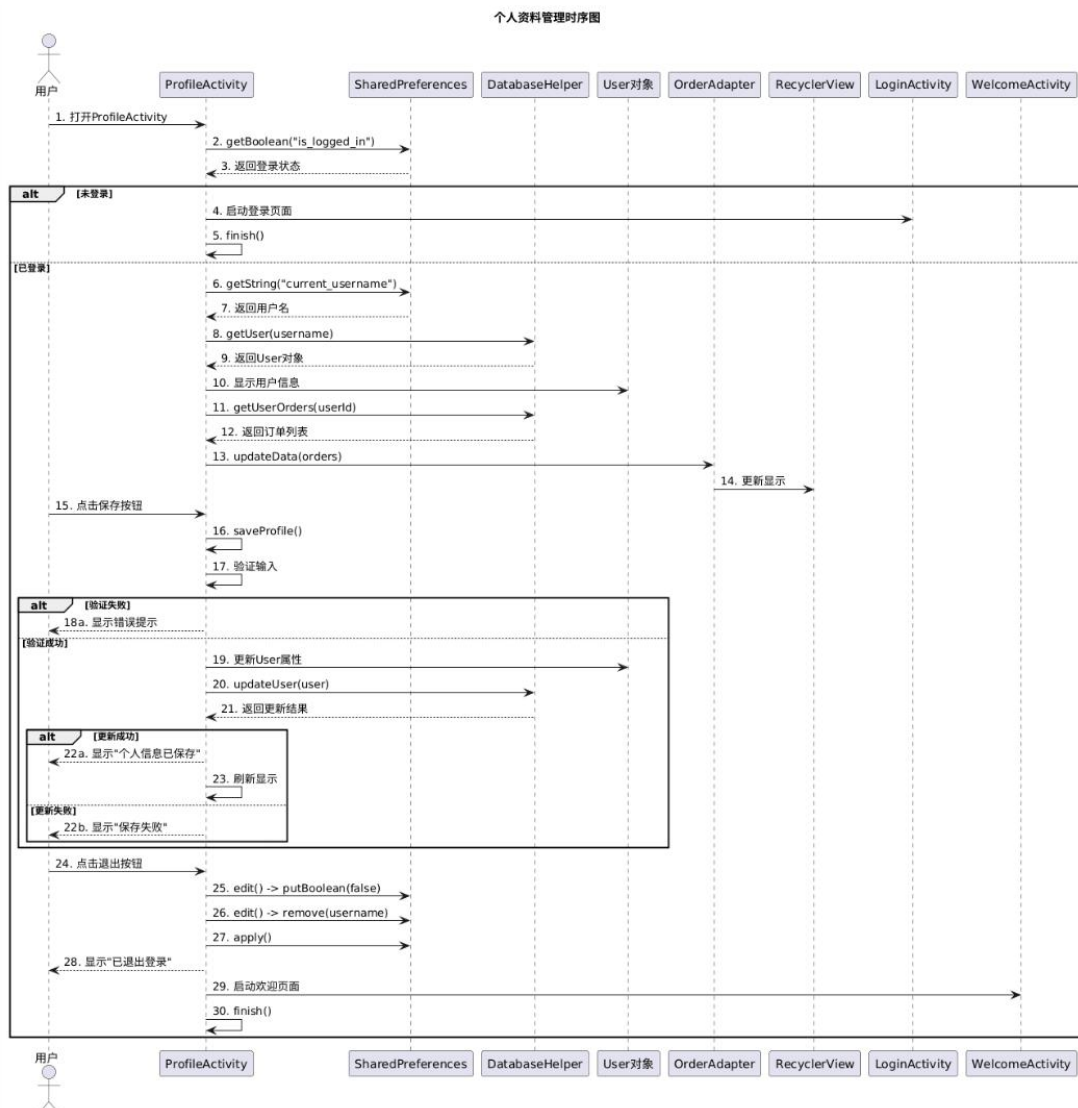
1 usage
private boolean isUserLoggedIn() { return sharedPreferences.getBoolean( key: "is_logged_in", defValue: false); }

1 usage
private void setupClickListeners() {
    btnLogin.setOnClickListener( View v -> loginUser());
    tvRegister.setOnClickListener( View v -> {
        startActivity(new Intent( packageContext: LoginActivity.this, RegisterActivity.class));
    });
}
```

```
57 private void loginUser() {
58     String username = etUsername.getText().toString().trim();
59     String password = etPassword.getText().toString().trim();
60
61     if (username.isEmpty() || password.isEmpty()) {
62         Toast.makeText( context: this, text: "请填写所有字段", Toast.LENGTH_SHORT).show();
63         return;
64     }
65
66     if (databaseHelper.checkUser(username, password)) {
67         // 保存登录状态
68         SharedPreferences.Editor editor = sharedPreferences.edit();
69         editor.putBoolean("is_logged_in", true);
70         editor.putString("current_username", username);
71         editor.apply();
72
73         Toast.makeText( context: this, text: "登录成功", Toast.LENGTH_SHORT).show();
74         startActivity(new Intent( packageContext: LoginActivity.this, MainActivity.class));
75         finish();
76     } else {
77         Toast.makeText( context: this, text: "用户名或密码错误", Toast.LENGTH_SHORT).show();
78     }
79 }
80 }
```

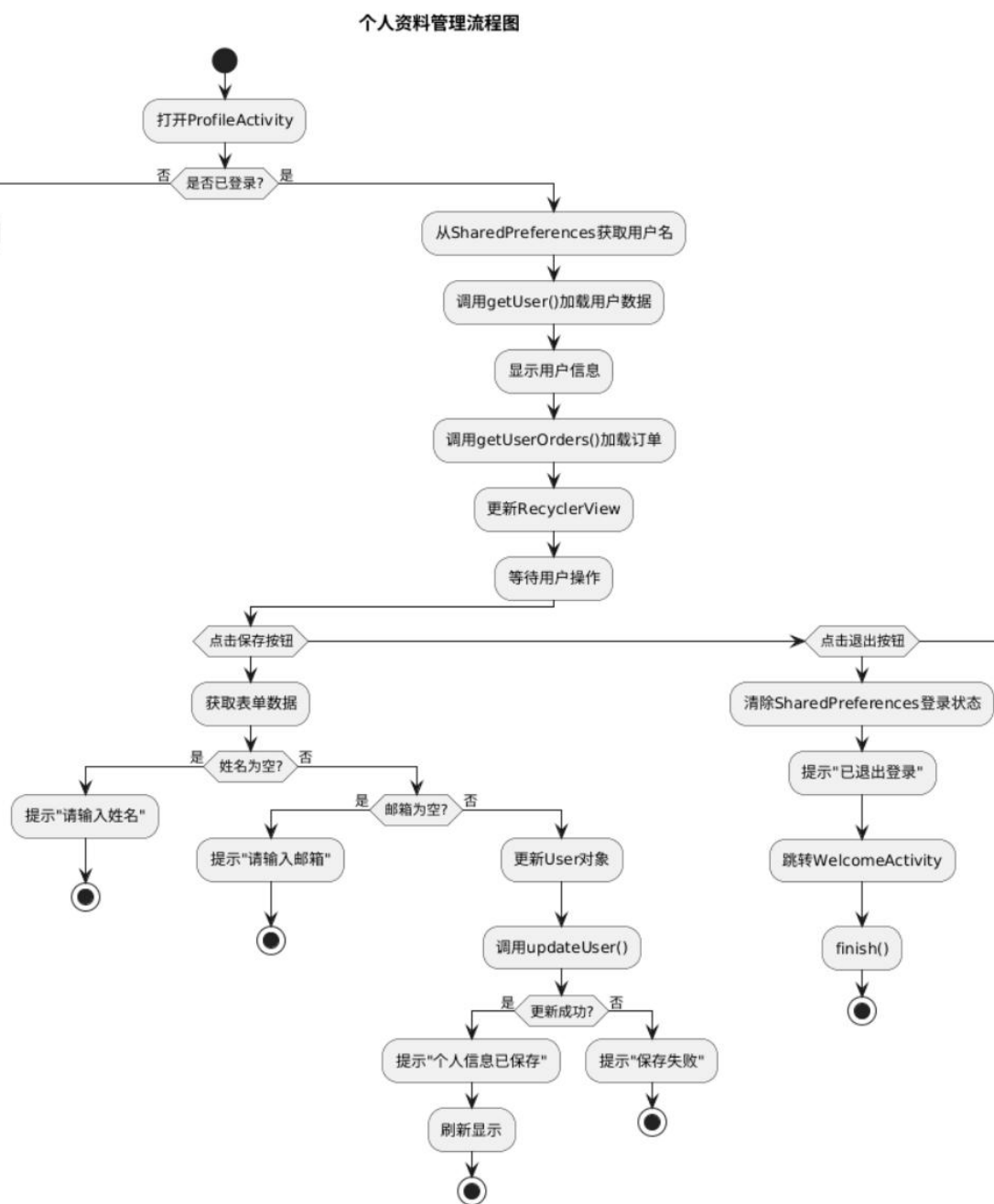
(3) 个人资料管理功能

①个人资料管理时序图

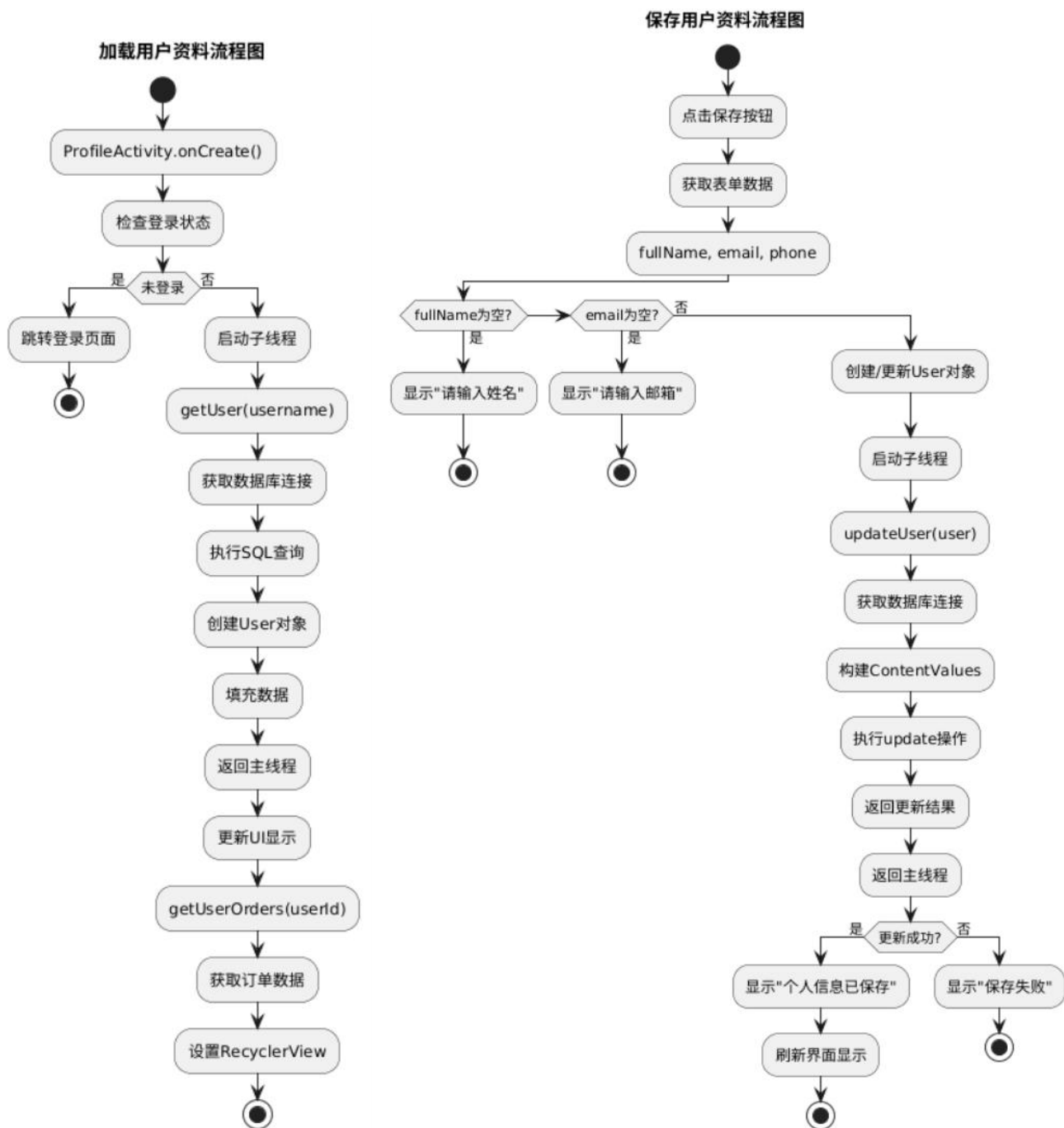


②个人资料管理流程图

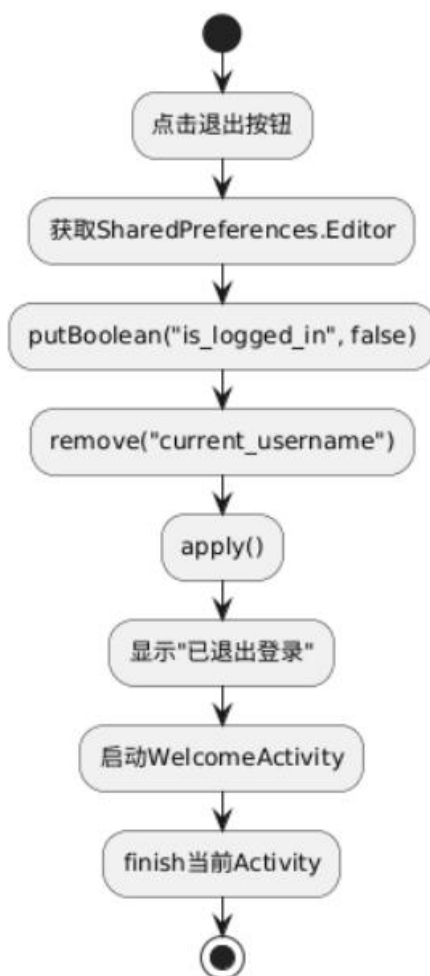
A.总图



B.子流程图



退出登录流程图



③核心代码

```
36  @+
37
38  protected void onCreate(Bundle savedInstanceState) {
39      super.onCreate(savedInstanceState);
40      setContentView(R.layout.activity_profile);
41
42      initViews();
43      databaseHelper = new DatabaseHelper( context: this);
44      sharedPreferences = getSharedPreferences( name: "user_prefs", MODE_PRIVATE);
45
46      // 检查登录状态
47      if (!isUserLoggedIn()) {
48          startActivity(new Intent( packageContext: this, LoginActivity.class));
49          finish();
50          return;
51      }
52
53      loadUserData();
54      setupRecyclerView();
55      setupClickListeners();
56      loadUserOrders();
57  }
```

```
1 usage
57 private void initView() {
58     ivUserAvatar = findViewById(R.id.ivUserAvatar);
59     tvUsername = findViewById(R.id.tvUsername);
60     tvUserEmail = findViewById(R.id.tvUserEmail);
61     etFullName = findViewById(R.id.etFullName);
62     etEmail = findViewById(R.id.etEmail);
63     etPhone = findViewById(R.id.etPhone);
64     btnSaveProfile = findViewById(R.id.btnSaveProfile);
65     btnLogout = findViewById(R.id.btnLogout);
66     rvOrders = findViewById(R.id.rvOrders);
67     tvNoOrders = findViewById(R.id.tvNoOrders);
68
69     // 设置返回按钮
70     if (getSupportActionBar() != null) {
71         getSupportActionBar().setDisplayHomeAsUpEnabled(true);
72         getSupportActionBar().setTitle("个人资料");
73     }
74 }

76 > private boolean isUserLoggedIn() { return sharedPreferences.getBoolean( key: "is_logged_in", defValue: false); }
79
2 usages
80 private void loadUserData() {
81     String username = sharedPreferences.getString( key: "current_username", defValue: "");
82     if (!username.isEmpty()) {
83         new Thread() -> {
84             currentUser = databaseHelper.getUser(username);
85             runOnUiThread() -> {
86                 if (currentUser != null) {
87                     displayUserInfo();
88                 }
89             });
90         }.start();
91     }
92 }
93
2 usages
94 private void displayUserInfo() {
95     if (currentUser == null) return;
96
97     tvUsername.setText(currentUser.getUsername());
98     tvUserEmail.setText(currentUser.getEmail());
99
100     etFullName.setText(currentUser.getFullName());
101     etEmail.setText(currentUser.getEmail());
102     etPhone.setText(currentUser.getPhone());
103 }
104
1 usage
105 private void setupRecyclerView() {
106     rvOrders.setLayoutManager(new LinearLayoutManager( context: this));
107     orderAdapter = new OrderAdapter(new ArrayList<>());
108     rvOrders.setAdapter(orderAdapter);
109 }
```

```
111     private void loadUserOrders() {
112         if (currentUser == null) return;
113
114         new Thread(() -> {
115             List<Order> orders = databaseHelper.getUserOrders(currentUser.getId());
116             runOnUiThread(() -> {
117                 if (orders != null && !orders.isEmpty()) {
118                     orderAdapter.updateData(orders);
119                     tvNoOrders.setVisibility(View.GONE);
120                     rvOrders.setVisibility(View.VISIBLE);
121                 } else {
122                     tvNoOrders.setVisibility(View.VISIBLE);
123                     rvOrders.setVisibility(View.GONE);
124                 }
125             });
126         }).start();
127     }
128
129     1 usage
130     private void setupClickListeners() {
131         btnSaveProfile.setOnClickListener( View v -> saveProfile());
132         btnLogout.setOnClickListener( View v -> logout());
133     }
134
135     private void saveProfile() {
136         String fullName = etFullName.getText().toString().trim();
137         String email = etEmail.getText().toString().trim();
138         String phone = etPhone.getText().toString().trim();
139
140         if (fullName.isEmpty()) {
141             Toast.makeText( context: this, text: "请输入姓名", Toast.LENGTH_SHORT).show();
142             return;
143         }
144
145         if (email.isEmpty()) {
146             Toast.makeText( context: this, text: "请输入邮箱", Toast.LENGTH_SHORT).show();
147             return;
148         }
149
150         if (currentUser != null) {
151             currentUser.setFullName(fullName);
152             currentUser.setEmail(email);
153             currentUser.setPhone(phone);
154
155             new Thread(() -> {
156                 boolean success = databaseHelper.updateUser(currentUser);
157                 runOnUiThread(() -> {
158                     if (success) {
159                         Toast.makeText( context: ProfileActivity.this, text: "个人信息已保存", Toast.LENGTH_SHORT).show();
160                         // 更新显示
161                         displayUserInfo();
162                     } else {
163                         Toast.makeText( context: ProfileActivity.this, text: "保存失败, 请重试", Toast.LENGTH_SHORT).show();
164                     }
165                 });
166             }).start();
167         }
168     }
```

```
169     private void logout() {
170         // 清除登录状态
171         SharedPreferences.Editor editor = sharedPreferences.edit();
172         editor.putBoolean("is_logged_in", false);
173         editor.remove( key: "current_username");
174         editor.apply();
175
176         Toast.makeText( context: this, text: "已退出登录", Toast.LENGTH_SHORT).show();
177         startActivity(new Intent( packageContext: this, WelcomeActivity.class));
178         finish();
179     }
180
181     1 usage
182     @Override
183     public boolean onSupportNavigateUp() {
184         onBackPressed();
185         return true;
186     }
187
188     @Override
189     protected void onResume() {
190         super.onResume();
191         // 刷新数据
192         loadUserData();
193         loadUserOrders();
194     }
```

4. 课程设计成果：实验报告、打包可运行安卓应用程序

4.1 运行界面展示

图 4-1 欢迎界面：显示汽车租赁 app 的 Logo 和欢迎语，可以选择登录或者注册。



图 4-3 注册界面：用户可以填写必要的信息进行账户注册。填写完成后点击注册，提示注册成功，会跳转到登录界面。

该图展示了应用程序的用户注册界面。界面顶部有一个汉堡菜单图标。标题“用户注册”位于顶部中央。下方是六个输入框，分别用于填写用户名、密码、确认密码、邮箱、手机号和姓名。每个输入框左侧都有相应的标签。用户名、密码和确认密码输入框中已输入了示例数据。在输入框下方是一个蓝色的“注册”按钮。整个界面背景为浅灰色，底部有一个白色的横线。

图 4-2 用户登录界面：用户可以输入用户名和密码进行登录。



图 4-4 应用主页界面：展示可租赁的汽车列表，每个汽车条目包含汽车品牌、型号和图片。



图 4-5 汽车详情界面：展示选定汽车的详细信息，包括品牌、型号、价格和描述。



图 4-6 预订界面：用户可以选择租赁的汽车、租赁时间和取车/还车地点进行预订。



图 4-7 订单确认界面：展示用户已选择的汽车、租赁时间和总价，并提供订单确认和支付选项。

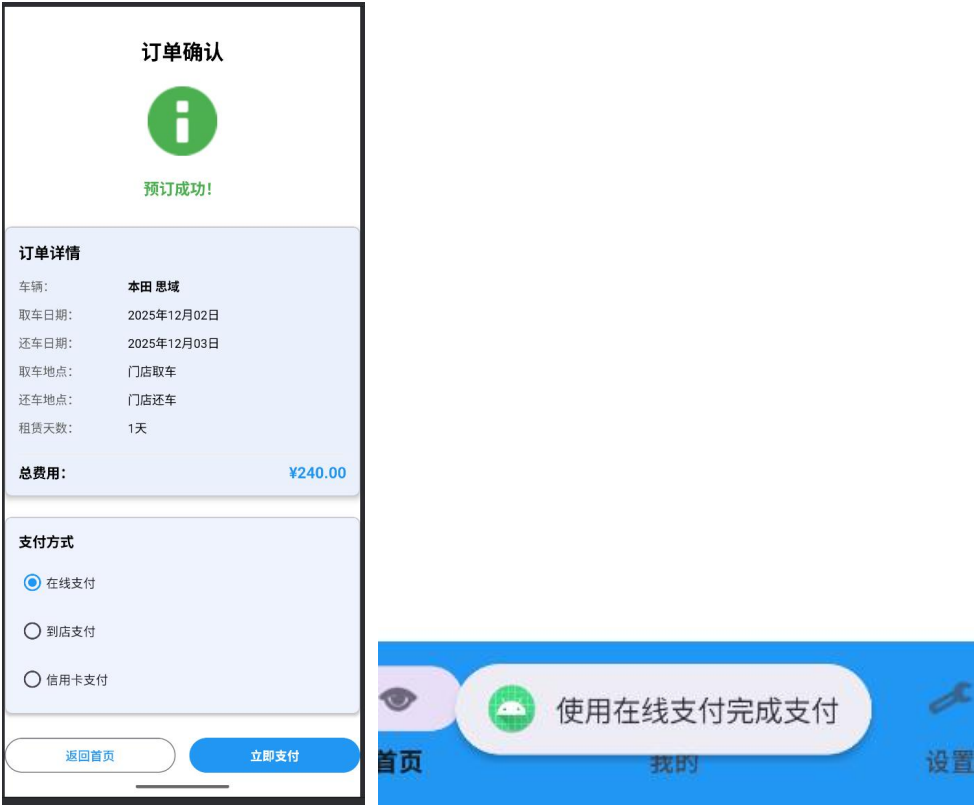


图 4-8 个人资料界面：显示用户的个人信息，包括头像、用户名和其他可选的详细信息。



图 4-9 设置界面：提供应用程序的设置选项，如通知设置、语言设置等。



5. 课程设计心得与体会

时光飞逝，为期两周的《移动应用开发课程设计》落下帷幕。当我看着自己亲手开发的“汽车租赁 APP”在手机上流畅运行时，心中涌起的是满满的成就感和深刻的感悟。这次课程设计不仅是对《移动应用开发》课程知识的一次全面检验，更是一次宝贵的工程实践经历，让我对 Android 开发有了全新的认识和理解。

从理论到实践的跨越是这次课程设计给我最深刻的体会。在课堂上，我们学习了 Activity 生命周期、SQLite 数据库、MVC 架构等理论知识，但只有当自己动手开发一个完整的 APP 时，才真正理解了这些知识点的实际应用价值。记得在实现用户注册功能时，最初我简单地认为就是几个 EditText 和一个 Button 的组合，但在实际编码中，我遇到了表单验证、密码一致性检查、用户名重复判断、数据库线程安全等一系列问题。这让我深刻体会到，理论知识只是基础，真正的能力体现在解决实际问题的过程中。

MVC 架构的实践应用让我受益匪浅。在项目初期，我对 MVC 的理解还停留在概念层面，但在设计和编码过程中，我逐渐体会到了这种架构模式的优势。当需要修改用户界面时，我只需在 View 层调整布局文件；当业务逻辑需要优化时，我可以在 Model 层独立修改；当需要增加新功能时，清晰的层次结构让我能够快速定位修改位置。特别是在实现个人资料管理功能时，

ProfileActivity 作为 Controller 协调着 View 的显示和 Model 的数据操作，这种职责分离的设计让代码维护变得更加容易。通过这次实践，我不仅掌握了 MVC 的实现方法，更理解了良好的软件架构对于项目可维护性的重要意义。

问题解决能力的提升是本次课程设计的重要收获。开发过程中，我遇到了许多意想不到的问题：从 Android Studio 的环境配置问题，到 SQLite 数据库版本升级导致的数据丢失，再到 RecyclerView 列表刷新不及时等。每一个问题的解决过程都是一次宝贵的学习机会。例如，在实现数据库操作时，我最初在主线程中执行查询导致界面卡顿，通过学习 Android 的多线程机制，我掌握了使用子线程执行耗时操作、通过 runOnUiThread 更新界面的正确方法。这些经验让我明白，在软件开发中，遇到问题并不可怕，关键是要有系统性的调试思路和解决问题的耐心。

团队协作与文档撰写也是这次课程设计给我的重要启示。虽然这是一个个人项目，但在与同学的交流讨论中，我学到了很多解决问题的不同思路。同时，撰写设计报告的过程让我认识到技术文档的重要性。清晰的文档不仅是对自己工作的总结，更是与他人沟通的桥梁。在整理代码、绘制流程图、编写说明文档的过程中，我锻炼了自己的表达能力和逻辑思维能力。

当然，这次课程设计也让我看到了自己的不足。在项目初期，由于对 Android 开发流程不够熟悉，我在需求分析和系统设计上花费了较多时间；在代码实现过程中，有时过于追求功能的完成而忽略了代码的优化和重构。这些都是我未来需要改进的地方。

展望未来，我希望能够在这个项目的基础上继续完善，比如增加网络通信功能实现真实的服务器交互，集成地图 API 提供更便捷的取还车服务，或者使用更先进的架构模式如 MVVM 进行重构。这次课程设计让我对移动应用开发产生了浓厚的兴趣，也为我后续的学习和实践打下了坚实的基础。

最后，我要衷心感谢指导老师的悉心指导和耐心解答，感谢同学们的热情帮助。这次课程设计不仅让我掌握了 Android 开发的实践技能，更重要的是培养了我面对复杂问题的分析能力和解决能力。这段经历将成为我大学生生活中宝贵的财富，激励我在计算机科学的道路上继续前行。

6. 课程设计附录 1：参考文献

[1]Android 官方文档.

[2]Android Studio 2025.1 官方帮助文档及内置代码模板.

[3]CSDN、博客园：查阅了部分关于界面适配、数据库设计的中文技术博客，获得了实践层面的启发

[4]Android Asset Studio：用于生成和定制部分图标资源.